Time-to-First-Spike Identity Mapping

刘沛雨

2025.6.8

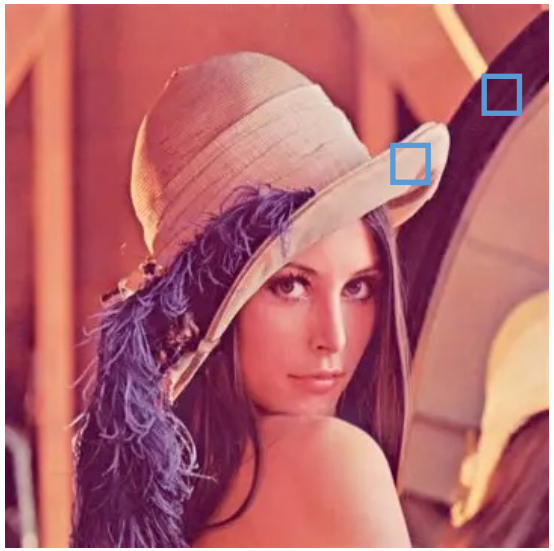
Outline

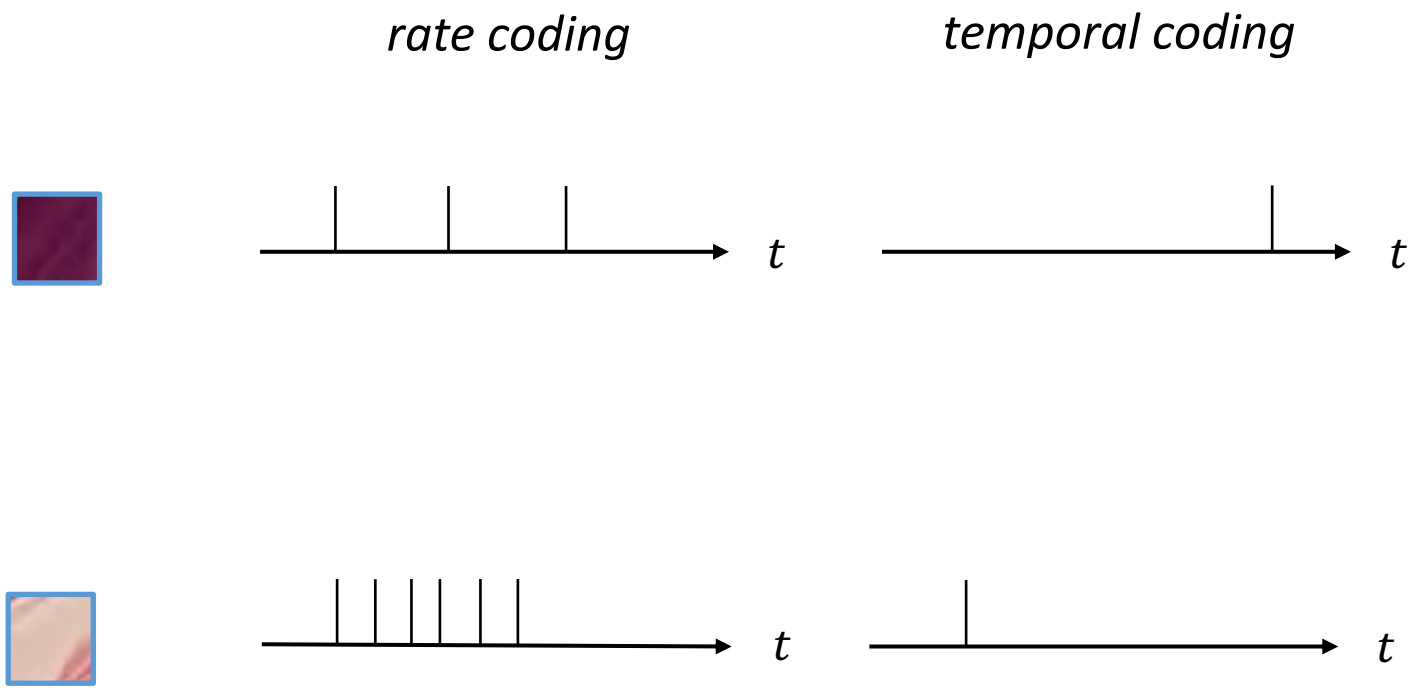
- Background
  - ✓ Rate coding & temporal coding
  - ✓ SNN Training
- Methods
  - ✓ Time-to-first-spike (TTFS) coding
  - ✓ Two-stage integrate-and-fire (IF) neuron
  - ✓ Identity mapping between ReLU and TTFS Neuron
  - ✓ Gradient flow analysis
- Results
  - ✓ Reproduction
- Discussion

How spikes encode information?

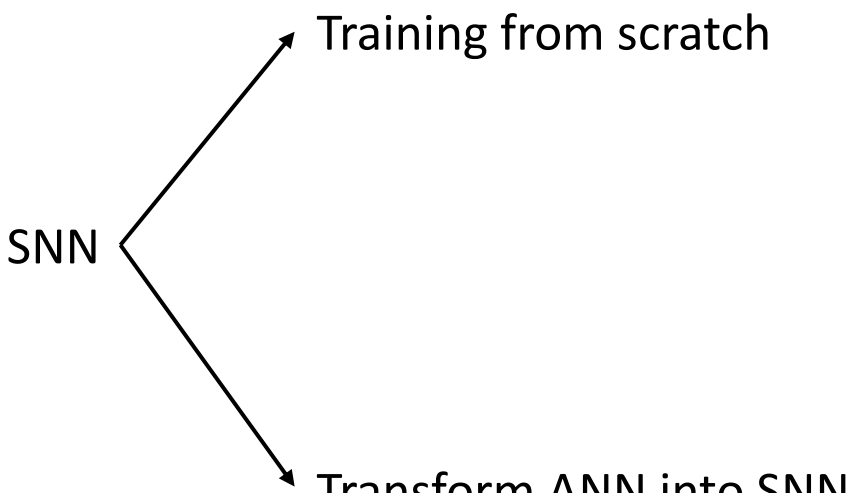
Spike itself does not contain any information

- Every spike looks similar in shape
- Frequency and timing are important
  - ✓ Through frequency – *rate coding*
  - ✓ Through timing – *temporal coding*





SNN Training



$\boldsymbol{s} = \Theta(\boldsymbol{v}(t) - \theta) \in \{0,1\}$

$\boldsymbol{s}$: spikes sent to next layer

Θ: Heaviside step function – non-differentiable at 0

θ: threshold

↓

surrogate gradient

- View spiking neurons as special activation functions
- Construct mapping between ANN and SNN
  - ✓ e.g., ReLU outputs ⟷ Firing rate/time

SNN Training

Article

https://doi.org/10.1038/s41467-024-51110-5

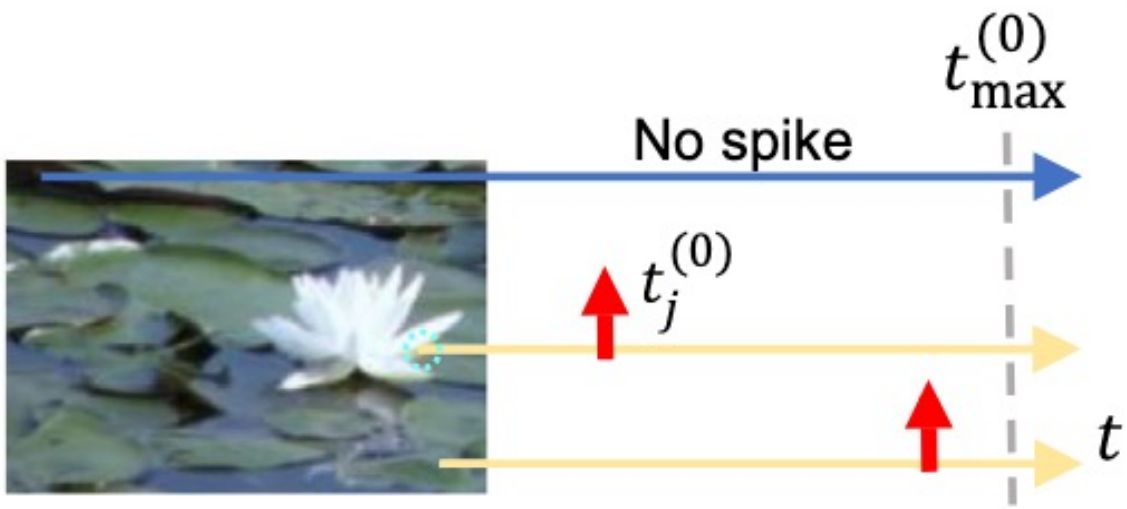
High-performance deep spiking neural networks with 0.3 spikes per neuron

Received: 20 November 2023

Accepted: 29 July 2024

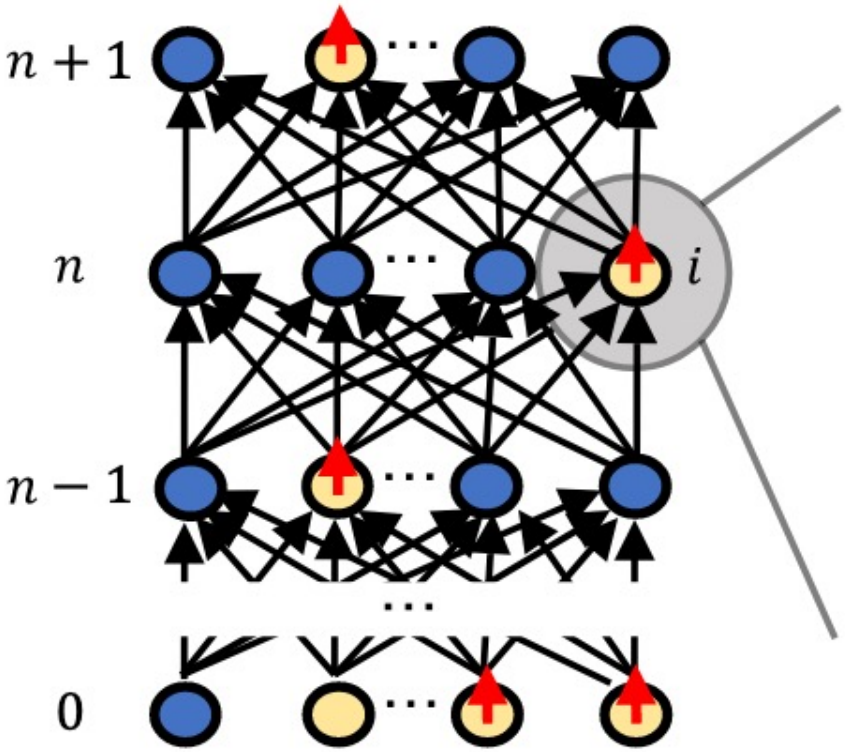
Ana Stanojevic[1,2], Stanisław Woźniak[1], Guillaume Bellec[2,3], Giovanni Cherubini[1], Angeliki Pantazi[1] & Wulfram Gerstner[2,3]

Time-to-First-Spike (TTFS) Coding

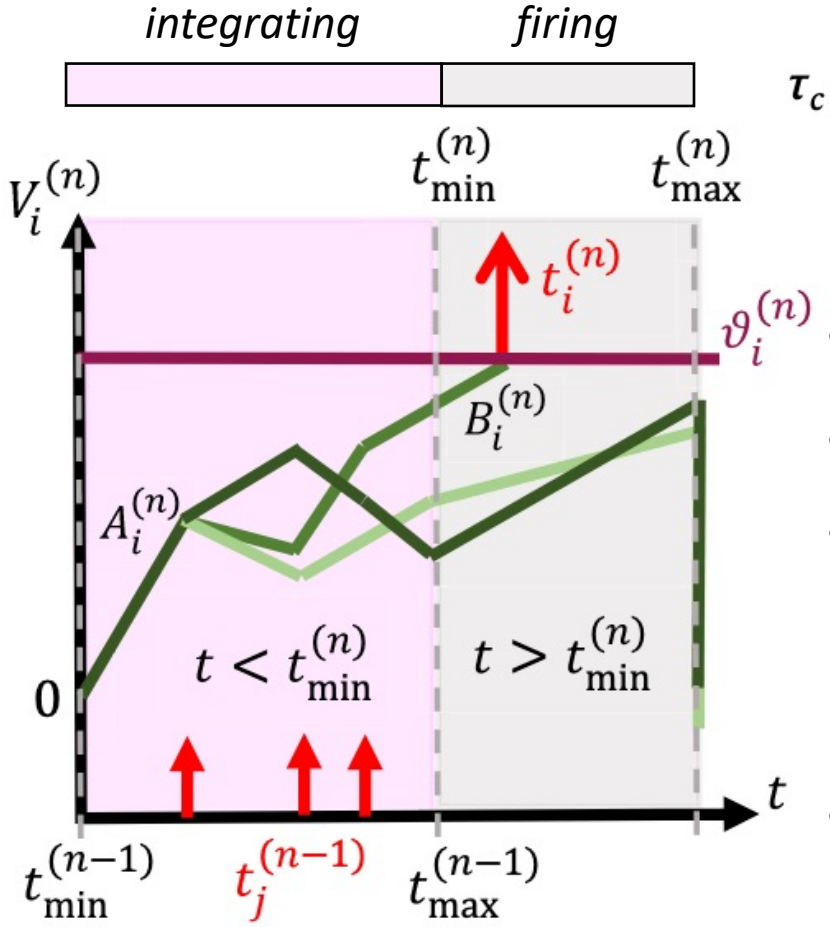


High intensity leads to early firing.

Two-stage IF Neuron



Feed-Forward Network

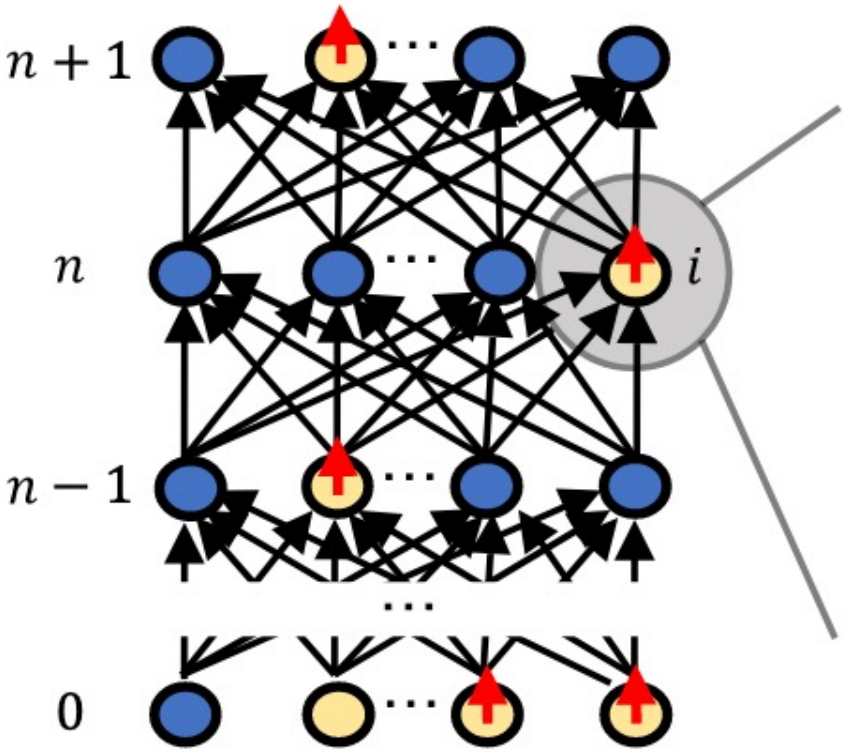


Neuronal Dynamics

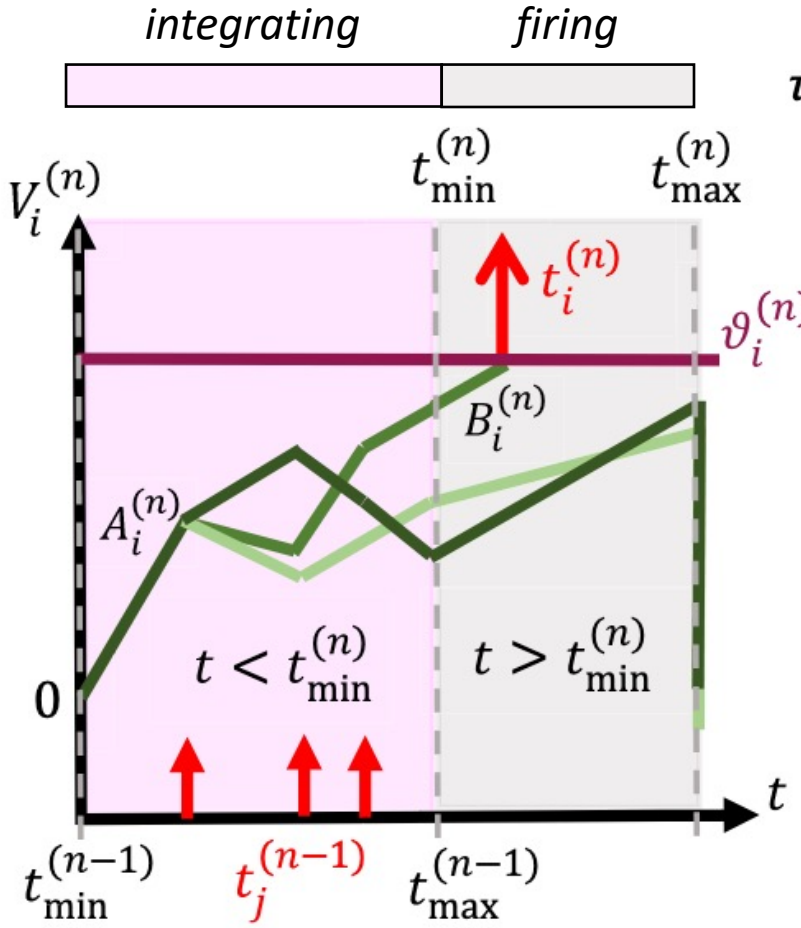
$$\tau_c \frac{\mathrm{d}V_i^{(n)}}{\mathrm{d}t} = \begin{cases} A_i^{(n)} + \sum_j W_{ij}^{(n)} H\left(t - t_j^{(n-1)}\right) \\ B_i^{(n)} \end{cases}$$

- Assume $\tau_c = 1$
- Initial slope: $A_i^{(n)}$
- After entering *firing* phase, potential grows constantly with $B_i^{(n)}$
- Each neuron fires at most once

Two-stage IF Neuron



Feed-Forward Network



Neuronal Dynamics

$$\tau_c \frac{\mathrm{d}V_i^{(n)}}{\mathrm{d}t} = \begin{cases} A_i^{(n)} + \sum_j W_{ij}^{(n)} H\left(t - t_j^{(n-1)}\right) \\ B_i^{(n)} \end{cases}$$

- Constraint #1: $t_{\min}^{(n)} = t_{\max}^{(n-1)}$
- Constraint #2: no neurons fire before $t_{\min}^{(n)}$ (through dynamic threshold) and after $t_{\max}^{(n)}$

$$\vartheta_i^{(n)} =^{\text{def}} \tilde{\vartheta}_i^{(n)} - D_i^{(n)}$$

Identity Mapping Between ReLU and TTFS Neuron

$$\tau_c \frac{dV_i^{(n)}}{dt} = \begin{cases} A_i^{(n)} + \sum_j W_{ij}^{(n)} H\left(t - t_j^{(n-1)}\right) \\ B_i^{(n)} \end{cases}$$

$$\vartheta_i^{(n)} =^{\text{def}} \tilde{\vartheta}_i^{(n)} - D_i^{(n)}$$

$$A_i^{(n)} = 0$$

$\Leftrightarrow$

$x_j^{(n)}$ (after ReLU) $\quad w_{ij}^{(n)} \quad x_j^{(n-1)} \quad bias_i^{(n)}$

$$\frac{t_{\max}^{(n)} - t_i^{(n)}}{\tau_c} = \sum_j \frac{W_{ij}^{(n)}}{B_i^{(n)}} \cdot \frac{t_{\max}^{(n-1)} - t_j^{(n-1)}}{\tau_c} + \frac{t_{\max}^{(n-1)} - t_{\min}^{(n-1)}}{\tau_c} - \frac{\vartheta_i^{(n)}}{B_i^{(n)}}$$

$t_i^{(n)}$: time when neuron i in layer n fires (assume it fires).

- Constraint #2: no neurons fire before $t_{\min}^{(n)}$ (through dynamic threshold) and after $t_{\max}^{(n)}$

A two-stage IF neuron fires in SNN $\Leftrightarrow$ Corresponding ReLU neuron produces a positive output in ANN

Main Findings - Gradients are still not equivalent.

- Pervious research used similar neuron models and methods to derive identity mapping
  - ✓ But only achieved equivalence at initialization
  - ✓ Problems must lie in backpropagation

$$x_j^{(n)}\ \text{(after ReLU)} \qquad w_{ij}^{(n)} \qquad x_j^{(n-1)} \qquad bias_i^{(n)}$$

$$\frac{t_{\max}^{(n)} - t_i^{(n)}}{\tau_c} = \sum_j \frac{W_{ij}^{(n)}}{B_i^{(n)}} \cdot \frac{t_{\max}^{(n-1)} - t_j^{(n-1)}}{\tau_c} + \frac{t_{\max}^{(n-1)} - t_{\min}^{(n-1)}}{\tau_c} - \frac{\vartheta_i^{(n)}}{B_i^{(n)}}$$

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\mathbf{W}^{(n)}} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\mathbf{V}^{(N+1)}} \frac{\mathrm{d}\mathbf{V}^{(N+1)}}{\mathrm{d}\mathbf{t}^{(N)}} \frac{\mathrm{d}\mathbf{t}^{(N)}}{\mathrm{d}\mathbf{t}^{(N-1)}} \cdots \frac{\mathrm{d}\mathbf{t}^{(n+1)}}{\mathrm{d}\mathbf{t}^{(n)}} \frac{\mathrm{d}\mathbf{t}^{(n)}}{\mathrm{d}\mathbf{W}^{(n)}},$$

$$\frac{\mathrm{d}\mathbf{t}^{(n)}}{\mathrm{d}\mathbf{t}^{(n-1)}} = M^{(n-1)} \cdot \frac{1}{B^{(n)}} \cdot W^{(n)} = M^{(n-1)} w^{(n)}.$$

$M^{(n-1)} = \mathrm{diag}\left\{m_1^{(n)}, m_2^{(n)}, \ldots, m_N^{(n)}\right\}, m_i^{(n)} \in \{0,1\}$ - *gradient gating*

$B^{(n)} = \mathrm{diag}\left\{B_1^{(n)}, B_2^{(n)}, \ldots, B_N^{(n)}\right\}, m_i \in \{0,1\}$

$w^{(n)}$: weights in ANN corresponding to $W^{(n)}$ in SNN

Main Findings - Gradients are equivalent only when $B^{(n)} = 1$.

- Pervious research set different $B_i^{(n)}$ for each neuron i in layer n
  - ✓ leading to “unequal” gradient flow
  - ✓ while for ReLU the gradient is always 1 under the same condition (firing ⟺ producing positive outputs)

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\mathbf{W}^{(n)}} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\mathbf{V}^{(N+1)}} \frac{\mathrm{d}\mathbf{V}^{(N+1)}}{\mathrm{d}\mathbf{t}^{(N)}} \frac{\mathrm{d}\mathbf{t}^{(N)}}{\mathrm{d}\mathbf{t}^{(N-1)}} \cdots \frac{\mathrm{d}\mathbf{t}^{(n+1)}}{\mathrm{d}\mathbf{t}^{(n)}} \frac{\mathrm{d}\mathbf{t}^{(n)}}{\mathrm{d}\mathbf{W}^{(n)}},$$

$$\frac{\mathrm{d}\mathbf{t}^{(n)}}{\mathrm{d}\mathbf{t}^{(n-1)}} = M^{(n-1)} \cdot \frac{1}{B^{(n)}} \cdot W^{(n)} = M^{(n-1)} w^{(n)}.$$

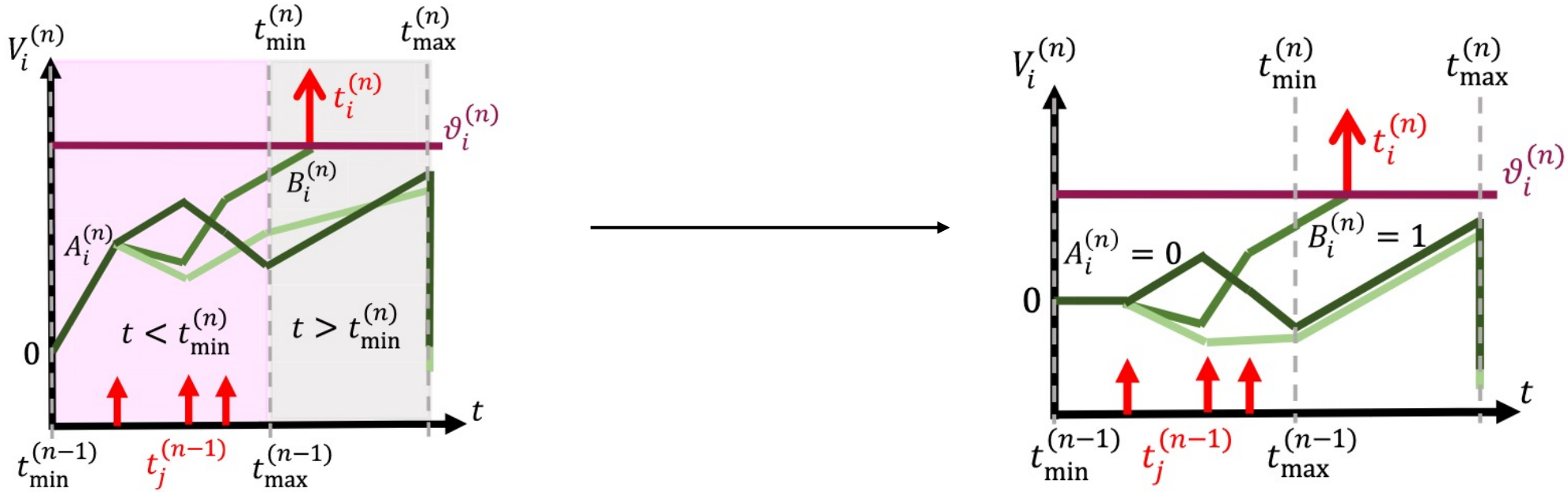
$M^{(n-1)} = \mathrm{diag}\left\{m_1^{(n)}, m_2^{(n)}, \ldots, m_N^{(n)}\right\}, m_i^{(n)} \in \{0,1\}$ - *gradient gating*

$B^{(n)} = \mathrm{diag}\left\{B_1^{(n)}, B_2^{(n)}, \ldots, B_N^{(n)}\right\}, m_i \in \{0,1\}$

$w^{(n)}$: weights in ANN corresponding to $W^{(n)}$ in SNN

Main Findings - Gradients are equivalent only when $B^{(n)} = 1$.

- Pervious research set different $B_i^{(n)}$ for each neuron i in layer n
  - ✓ leading to “unequal” gradient flow
  - ✓ while for ReLU the gradient is always 1 under the same condition (firing ⟺ producing positive outputs)
  - ✓ thus setting $B^{(n)}$ to 1



Performance on MNIST

Model	Arch.	Test acc. [%] (from paper)	Test acc. [%] (reproduced)
ReLU	FC2	98.30	98.38
TTFS	**FC2**	**98.30**	**98.40**
ReLU	FC16	98.43 ± 0.07	98.14
TTFS	**FC16**	**98.43 ± 0.07**	**98.16**
ReLU	VGG16	99.57 ± 0.01	99.59
TTFS	**VGG16**	**99.58 ± 0.01**	**99.50**

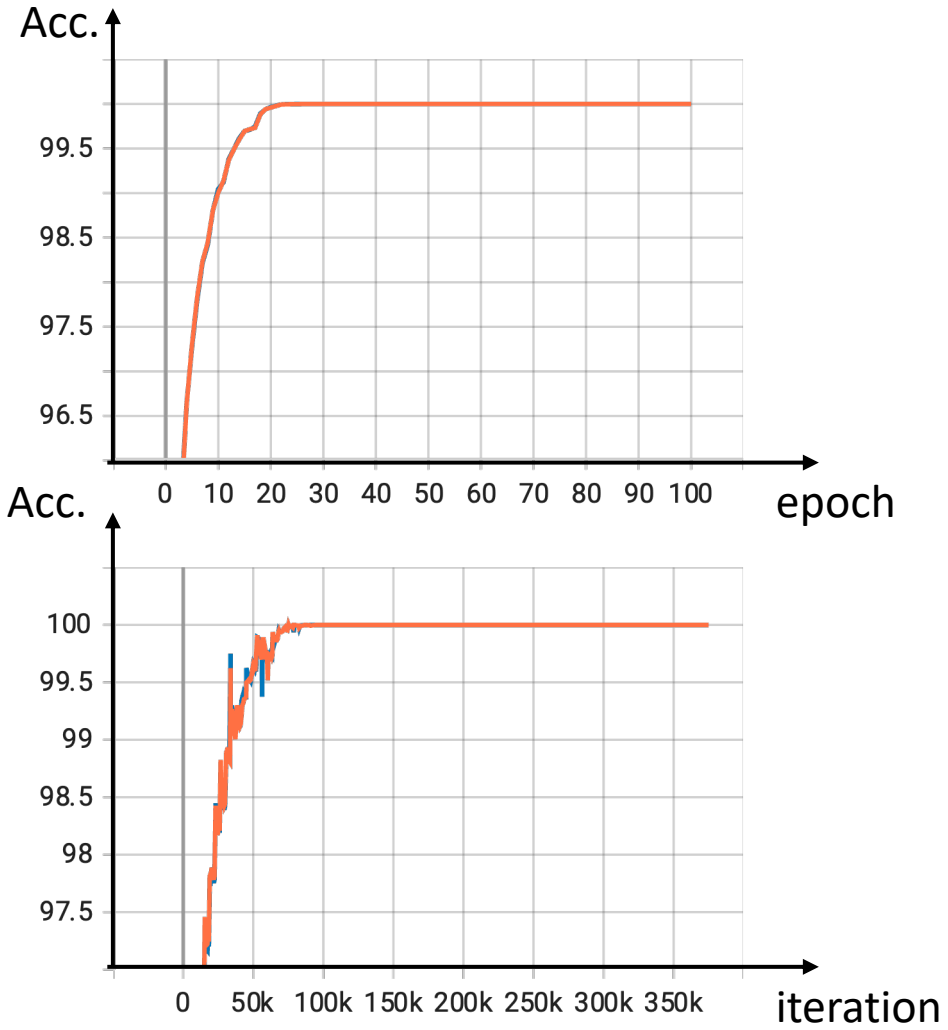
Performance on FashionMNIST

Model	Arch.	Test acc. [%] (from paper)	Test acc. [%] (reproduced)
ReLU	FC2	90.14	90.22
TTFS	**FC2**	**90.14**	**90.31**
ReLU	LeNet5	90.91 ± 0.17	90.73
TTFS	**LeNet5**	**90.94 ± 0.25**	**90.80**

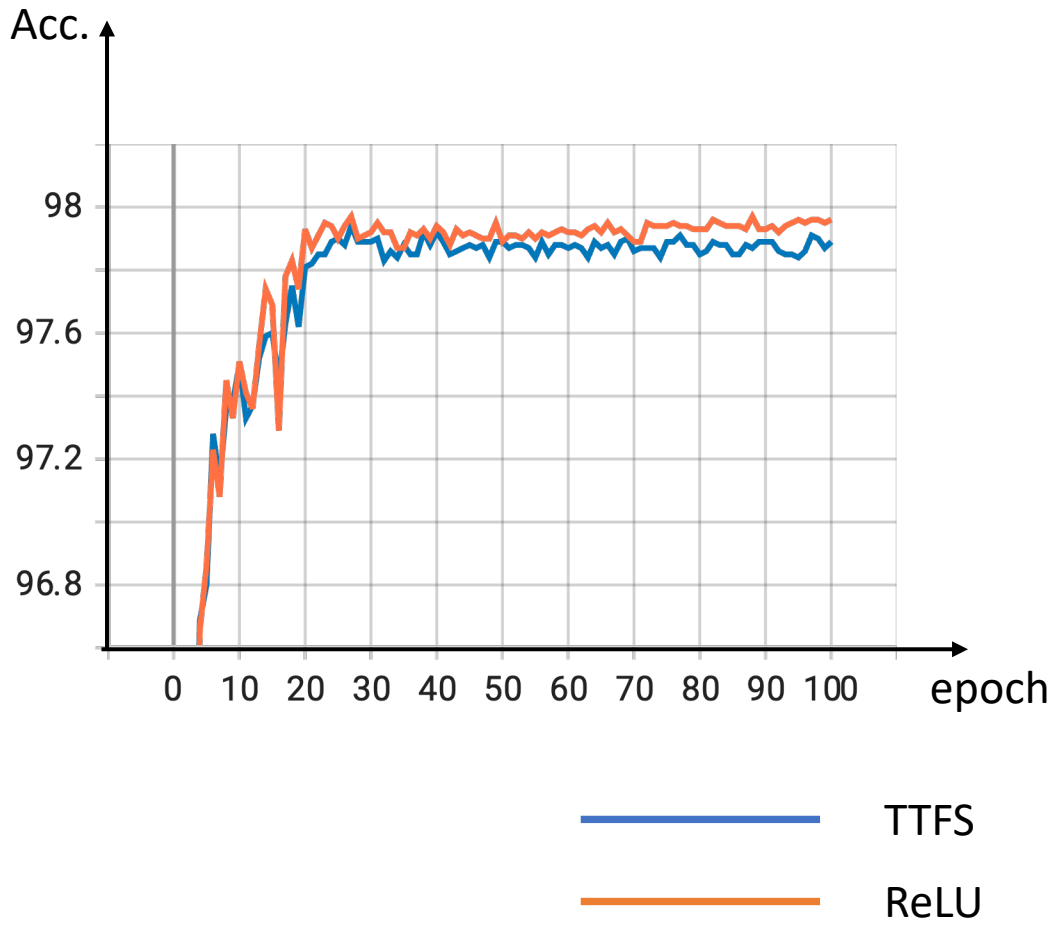
Implementation is based on PyTorch; Codes will be released on GitHub

Training Equivalence (demo. on a 8-layer MLP)

train



test



Discussion

Summary

- Potential in the *firing* phase scales the gradients during backpropogation
- Always make sure that these potential has same gradient with that of ANN activation functions

Future work

- Reproduction
  - ✓ Visualization
  - ✓ Testing on larger benchmarks (e.g., CIFAR10/100, ImageNet, PLACE365, etc.)
  - ✓ Robustness tests
  - ✓ ANN2SNN conversion in TTFS
- Other possible directions
  - ✓ Implementing TTFS on ResNet
  - ✓ Better Compatibility with BatchNorm

Reference

- Stanojevic A. *et al.* High-performance deep spiking neural networks with 0.3 spikes per neuron. *Nat Commun.* 2024 Aug 9;15(1):6793.
- Stanojevic A. *et al*. An exact mapping from ReLU networks to spiking neural networks. *Neural Netw.* 2023 Nov;168:74-88.

Thank you!

Q&A